

LinTeX: Enhancing L^AT_EX document accessibility for screen reader users

by
Mohammad Danyal

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project



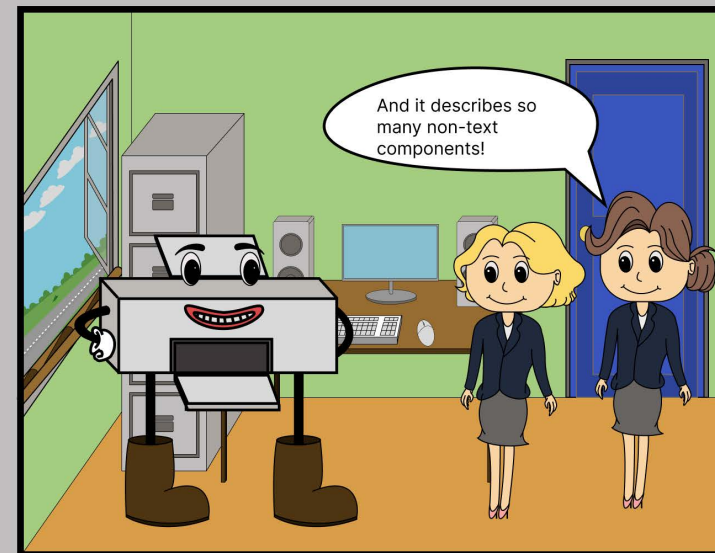
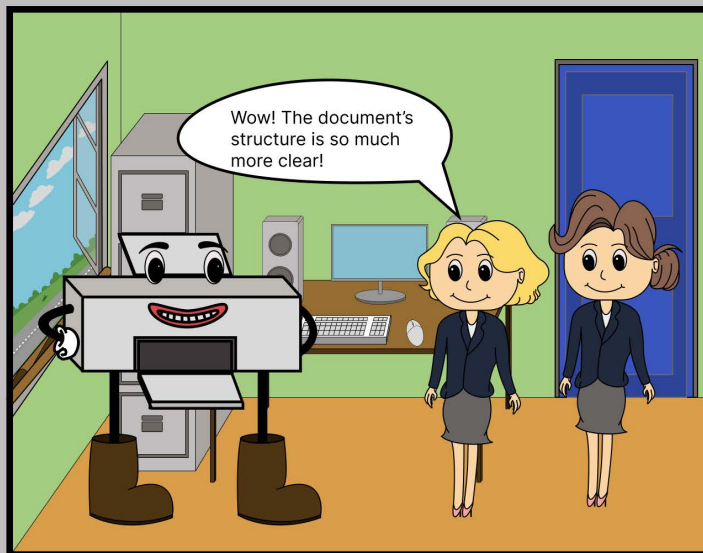
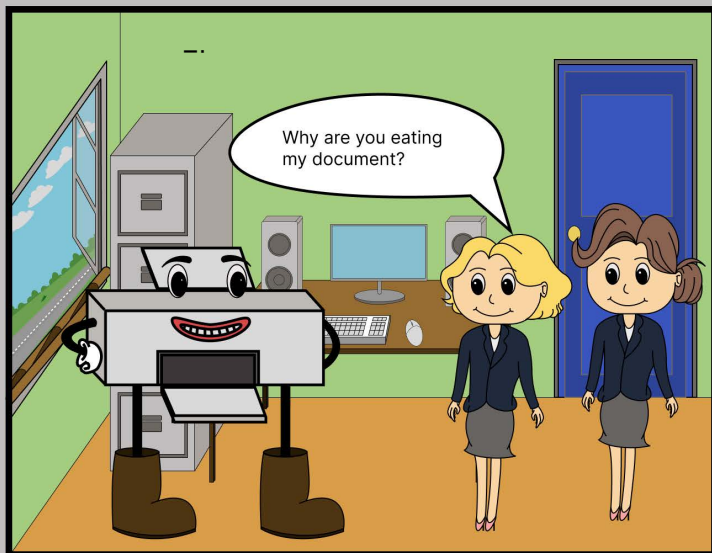
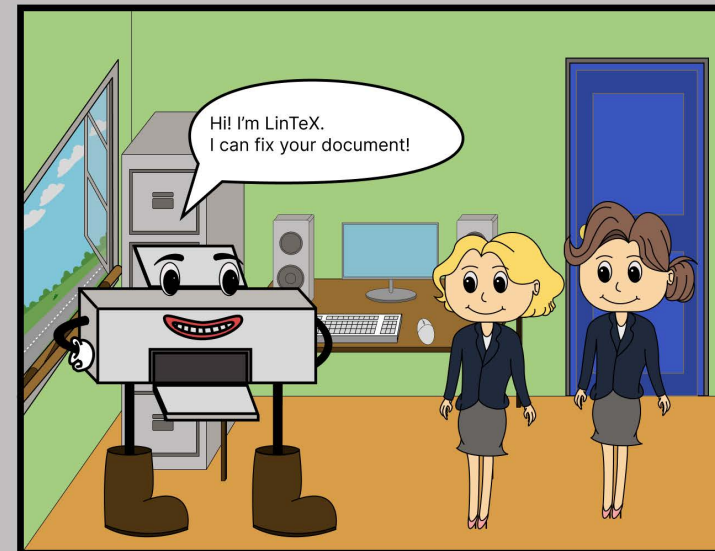
Department of
Computer and Information Sciences

April, 2026

Abstract

The abstract is a bird's eye view of the project.

It should not exceed one page. Mention the scope and objectives of the project, the methodology used, the main findings, and significance of your results.



Acknowledgements

I would like to thank my supervisor, Brian Mitchell, for his continuous guidance and support throughout the project. His depth of knowledge in related topics and the management of the project overall had vastly improved the project outcome. Although Brian would likely say luck is just a probability taken personally, I consider myself lucky to have come across and taken an interest in his project description.

I would like to provide further thanks to my friends and family who helped provide advice and support throughout the critical periods of the project. Their support was invaluable.

Finally, I would like to give thanks to the anonymous participants who went through the trouble of attending my user study, despite the hassles of it being an in-person session.

Contents

Overview Diagram	i
Acknowledgments	ii
Glossary	v
1 Introduction	1
1.1 Identifying L ^A T _E X's key accessibility problems for screen reader users	1
1.2 Why accessibility is important	1
1.3 Why LuaLaTeX provides better accessibility than PDFLaTeX	1
1.4 Creating LinTeX	1
1.5 aims and objectives	1
1.6 Limitations of the project overall	1
1.7 research questions/hypothesis	1
1.8 from nov progress	2
2 Background Literature	4
2.1 Choice of programming language	4
2.2 Choosing a PEG library under the Go programming language	5
2.3 Choice of frameworks and libraries	5
2.4 Guidelines for accessibility	7
2.5 Choosing a software development methodology	8
2.6 Screen reader choice	9
2.7 User study design choices	10
2.8 GUI design considerations	11
3 Specification & Design	12
3.1 Methodology	12
3.2 Analysis	12
3.3 Requirements	12
3.4 Design	13
4 Implementation	15
4.1 general overview of technologies used	15
4.2 website with server-client relationship	15
4.3 possibly include things about peg	15
4.4 verification & validation	15
5 User study	16
5.1 Study design	16
5.2 Hypothesis	16
5.3 Results	17
5.4 Analysis	20
5.5 Hypothesis testing and discussion	23
6 Discussion and reflection	25

6.1 something	25
7 Conclusion	26
7.1 Project summary	26
7.2 Project limitations	26
7.3 Achievements – potentially optional/swap with final thoughts or another area?	26
7.4 Future work	26
7.5 Reflection	26
7.6 Final thoughts	26
A Appendix	29
A.1 Ethical approval form	29
A.2 Participant information sheet	29
A.3 Consent form	29
A.4 User study script	29

List of Figures

1 Comparing screen reader output quality ratings before and after applying LinT _E X	18
2 Paired t-test formula	19
3 Participants likelihood to support screen reader users in the future	20

List of Tables

2 Languages and related PEG frameworks	4
3 Languages and related website development support	5
4 Comparison between PEG and Pigeon	6
5 Screen reader pricing status and OSes	9
6 Memorable aspects of document extracts	17
7 Difficult aspects from the initial and final listenings	18
8 Simple aspects from the initial and final listenings	19
9 LinT _E X usability	21

Glossary

accessible document	an accessible document created applying LinT _E X to the inaccessible document. 8, 16, 17, 22, 23
alternative text	text used in place of non-text components for accessibility purposes. 8
AST	abstract syntax tree 6
BDD	behaviour-driven development 8, 9
CDN	content delivery network 7
CGI	Common Gateway Interface 4
CLI	command-line interface 7
CMD	command prompt for Windows 7
CSS	Cascading Style Sheet 7
GUI	graphical user interface iii, 11
HTML	Hypertext Markup Language 6, 7
HTTP	HyperText Transfer Protocol 7
IDE	integrated development environment 11
inaccessible document	a L ^A T _E X document that has intentionally been made inaccessible for screen reader users. v, 11, 16, 17, 22, 24
JS	JavaScript 7
likert scale datum	a scale, usually with 5 or 7 points, indicating a participant's agreement or disagreement with some topic (Sullivan 2013). 10, 11
metadata	data about data, such as a file's structure. 18, 22
npm	Node package manager 7
OS	operating system iv, 9
PDF	Portable Document Format 6
PEG	parsing expression grammar iii, iv, 4–6, 8
qualitative data	data that cannot be represented numerically, such as descriptions, statements or quotes (National Library of Medicine 2026a). 10
quantitative data	data that can be represented in numerical form (National Library of Medicine 2026b). 10

right-tailed test	a test for whether the hypothesis should be rejected (Exeter 2025). 23
TDD	test-driven development 8, 9
think-aloud session	users think aloud while performing a given task, with the goal of identifying areas of improvement in the usability of the given context. 10, 11
UI/UX	user interface and user experience 11
WSL	Window's Subsystem for Linux 6

1 Introduction

1.1 Identifying $\text{\LaTeX}$'s key accessibility problems for screen reader users

Similar to a tool like Microsoft Word, $\text{\LaTeX}$ is a tool used to process documents. However, $\text{\LaTeX}$ does this in a programmatical manner, likening it to a programming language, whereas Microsoft Word has a that

- xetex - came after pdflatex but before luatex created for better font handling - specify not under consideration - since latex devs focused on luatex - was useful, but not used much - 'could be considered evolutionary deadend'/might become one/find authoritative source to say it is
- introduce latex here? - rough definition ig, and purpose - (intended to be math textbook)
- (small paragraph)

1.2 Why accessibility is important

- who would benefit from it - stats with disabilities e.g. how many people are blind and use screen readers - (use good sources e.g. government websites or world health organisations)
- severity and frequency are considerations

1.3 Why LuaLaTeX provides better accessibility than PDFLaTeX

1.4 Creating LinTeX

- intended purpose - (this section has the finished overview diagram)

1.5 aims and objectives

1.6 Limitations of the project overall

1.7 research questions/hypothesis

This is a concise and clear overview of your dissertation (more or less 2-3 pages). You can start off the project description provided of the project that was allocated to you and flesh it out.

Include (1) the problem you have tackled, (2) why this problem is worth addressing, (3) what you did to address it - in broad terms. Detail will come later.

i.e., issue(s) on which the research will focus, shall be clearly identified and described. You shall refer to past research work relevant to the topic and objectives, i.e., of the study. You shall outline where applicable the potential research output with respect to research transfer and uptake by the community.

The introduction says: this is an overview of the project. This is why I did it (the problem) and how I tackled it. It is the *runway* into the project. It lets the marker know what to expect of your report.

If you're doing a research project, this would be the place to include the research questions you plan to address. For example:

RQ1: Where did James Bond come from?

RQ2: Why are pumpkins orange?

If you're doing a project of type 1, include a list of objectives. For example:

Objective 1: Provide software to allow James Bond to become invisible.

Objective 2: Provide software to keep track of all loyalty points in one place.

Provide a 'map' of your dissertation. For example: Section 2 reviews the background literature that was reviewed to inform this project. Then Section ??.....

1.8 from nov progress

There is a lack of accessibility with $\text{\LaTeX}$, largely due to usage of outdated PDF $\text{\LaTeX}$ as opposed to Lua $\text{\LaTeX}$. Usage of this outdated $\text{\TeX}$ engine necessitates workarounds for desired features and accessibility in resulting PDF documents.

What can be done to make $\text{\LaTeX}$ documents more accessible?

Lin $\text{\TeX}$ is a developing tool dedicated to combat this problem by focusing on assisting screen readers' abilities to read the PDF documents produced from $\text{\LaTeX}$ documents. It aims to cover this area through suggesting structured tagging throughout a document, suggesting

components where it may be lacking. Additionally, it will request users to input appropriate alternative text ('alt text') for non-text components of the document, including figures, images, and tables. Further focus may be placed on features such as ensuring appropriate contrast in any colours in the document, appropriate Sans Serif fonts, and flexible document design, largely depending on what development time allows for.

Additionally, as a tool focused on promoting accessibility, it too should be accessible. Therefore, the tool will ensure strict adherence to the Web Content Accessibility Guidelines 2 (WCAG 2)

Why should I use LinTeX?

While the tool is not limited strictly to the field, it is aimed to assist the many academics that find difficulties in accessing reports and academic literatures. By utilising this tool, you can ensure a wider demographic of users can view your documents, without necessitating an especially large amount of research in accessibility; it has been done for you.

2 Background Literature

2.1 Choice of programming language

There were many considerations when deciding what programming language to write LinT_EX in. The languages chosen were based on those that I had prior experience in, to ensure minimal hindrance from the language itself to the development of the project. An important factor is ensuring LinT_EX's maintainability in the future. Therefore, only the languages that have actively maintained parsing expression grammar (PEG) libraries will be considered, as can be seen in Table 2. The most recent update times provided in Table 2 are in reference to 24th November 2025.

Table 2: Languages and related PEG frameworks

Language	PEG library link	PEG library is in development
Python	https://pythonhosted.org/pyPEG2/	No, last updated in October 2015
Python	https://github.com/erikrose/parsimonious	Yes, last updated in November 2025
JavaScript	https://github.com/pegjs/pegjs	No, last updated in November 2019
JavaScript	https://peggyjs.org/	Yes, last updated in September 2025
Go	https://github.com/pointlander/peg	Yes, last updated in November 2025
Go	https://github.com/mna/pigeon	Yes, last updated in November 2025
C	N/A	N/A
Java	https://github.com/sirthias/parboiled	Yes, last updated in November 2025
C#	https://code.google.com/archive/p/peg-sharp/	No, last updated in May 2011
C#	https://github.com/otac0n/Pegasus	No, last updated in December 2023
C++	https://github.com/yhirose/cpp-peglib	Yes, last updated in July 2025
PHP	https://github.com/hafriedlander/php-peg	No, last updated in May 2014
TypeScript	https://github.com/metadevpro/ts-pegjs	No, last updated in November 2024
TypeScript	https://github.com/EoinDavey/tsPEG	Yes, last updated in November 2025
Dart	https://github.com/darmie/dart-peg	No, last updated in July 2015

Based on the current criteria, we can rule out C, C#, PHP, and Dart.

As many languages remain, there is a need for stricter criteria: suitability to be utilised as a web application as well as compatibility with the University of Strathclyde's development web server (devweb) system.

It should be noted that while Go, C, C#, and C++ are not directly compatible with devweb, as referenced in Table 3, they can be supported through the use of Common Gateway Interface (CGI) scripts, and consequently should not be fully disregarded for this alone. However, based on their support for web development, C, Java, C#, and C++ will have a lower priority for being a chosen language.

Table 3: Languages and related website development support

Language	Web development support	DevWeb support
Python	Yes	Yes
JavaScript	Yes	Yes
Go	Yes	No
C	No, except for rare exceptions	No
Java	Yes, but not generally used	Yes
C#	Yes, but not generally used	No
C++	Yes, but not generally used	No
PHP	Yes	Yes
TypeScript	Yes	Yes
Dart	Yes	Yes

This leaves us with a focus placed on JavaScript, Go, and TypeScript. However, JavaScript will be removed from consideration as a strictly typed language is preferred for our context. Therefore, leaving Go and TypeScript as the main considerations for this project. Finally, as I have more experience and a stronger preference towards Go as opposed to TypeScript, it was the chosen programming language for the project.

2.2 Choosing a PEG library under the Go programming language

Go has two libraries that are used in relation to PEG systems: Pigeon by Martin Angers (mna) and PEG by Andrew Snodgrass (pointlander). Direct comparisons with core components of both have been made in Table 4.

While examples are relatively similar, the simplicity of installation for Pigeon as opposed to PEG, combined with the depths of documentation both in generated code and outwith it, makes Pigeon a more ideal option. Additionally, its greater number of features gives $\text{LinT}_{\text{E}}\text{X}$ more to make use of. Therefore, Pigeon was the chosen library for the project.

2.3 Choice of frameworks and libraries

2.3.1 Backend frameworks and libraries

There were three options for web frameworks and packages, namely Gin (Martínez-Almeida 2014), Gorilla (The Gorilla Authors 2023), and net/http (The Go Authors 2009c). For $\text{LinT}_{\text{E}}\text{X}$, net/http was chosen for the key reason that it had the least barriers to entry, as there was a provided guide on the Go development website that was utilised in the

Table 4: Comparison between PEG and Pigeon

	Pigeon	PEG
Installation	Requires running a <code>go install</code> command and setting up environment variables.	Requires supporting bash scripting (through Window's Subsystem for Linux (WSL) for Windows users), as well as running a <code>go install</code> command, cloning the repository, and running <code>go generate && go build</code> .
Documentation	Contains a Godoc page, wiki page, an examples subdirectory, and a detailed README.md providing what PEG features are supported, command line interface usage, installation, and examples.	Contains a detailed README.md and a PEG file syntax file containing what PEG features are supported, installation steps, and example projects links.
Generated Code	Generated Go code can be used to configure settings on the parser, view statistics or error information, debug, enable or disable memoization, and parse readers, files, and strings. Additionally, the generated code is well documented, with several comments surrounding usage of public functions.	Generated Go code can be used to run the PEG script on a provided string, debug and output the abstract syntax tree (AST), output errors in parsing, and enable or disable memoization.
Examples	Provides three direct examples: detecting indentation in files, parsing JSON files, and a calculator example. Examples were kept short (regarding PEG scripts) with the calculator and indentation examples showcasing usage via a main function.	Provides two project links for examples on PEG usage: a natural date/time parser and a scientific names parser. Both projects have large PEG scripts, with most lines being simple to read. The natural date/time project was intuitive to use and understand program flow.

project setup. However, in hindsight, more consideration towards what web framework to use would have had a great impact on the project as a whole, as net/http is a lower level web framework than Gin or Gorilla, consequently requiring more management on $\text{Li}\text{T}_{\text{E}}\text{X}$'s end. Some examples of difficulties that occurred are a lack of POST requests being easily supported and necessitating that server files are explicitly stated as opposed to implicitly defined within a given context.

The `html/template` (The Go Authors 2009b) package was included in the project. This package provided the ability to load Hypertext Markup Language (HTML) files through Go, while allowing information to be provided to them. The alternative choice was to provide a raw print output to the client, which made the package a far clearer winner.

For the project to be able to compile $\text{L}\text{A}\text{T}_{\text{E}}\text{X}$ documents to convert them into PDFs, it was necessary to either use a pre-built package or to compile $\text{L}\text{A}\text{T}_{\text{E}}\text{X}$ documents locally. Pre-built packages that were found include `go-latex` (T. I. Authors 2020) and `latex-fast-compile` (Kpym 2020). It was necessary to rule out `go-latex` as it emulated a $\text{L}\text{A}\text{T}_{\text{E}}\text{X}$ compiler, rather than directly linking to one. Additionally, it lacked documentation and necessary features,

making it unsuitable. Latex-fast-compile was also unsuitable due to using PDF $\LaTeX$ rather than Lua $\LaTeX$, which was desired.

Consequently, it became necessary to compile $\LaTeX$ documents directly. This was done through the use of command prompt for Windows (CMD), which allowed the 'lualatex' command to be ran, consequently compiling the requested file. It did, however, prerequisite that the hosting server had a $\LaTeX$ compiler in their backend, for the relevant Lua $\LaTeX$ engine. The os/exec (The Go Authors 2009a) package allowed for a CMD shell to be created within Go code, consequently allowing the 'lualatex' command to be run. Hence, compiling the $\LaTeX$ document that the user provides.

2.3.2 Frontend frameworks and libraries

As plain Cascading Style Sheet (CSS) is difficult to work with and maintain, it was necessary to decide on a CSS framework for the project. My prior experience in Bootstrap (T. B. Authors 2011) made it a close choice, but the opportunity to learn to utilise Tailwind CSS (Tailwind Labs 2017) was more enticing and eventually won the decision.

To implement Tailwind CSS, it was necessary to install its command-line interface (CLI) through Node package manager (npm). This permitted usage of Tailwind CSS without necessitating JavaScript (JS) within the codebase, and avoided workarounds such as a content delivery network (CDN). Additionally, due to difficulties in using Tailwind CSS, it was necessary to utilise a component library built on top of it, for which Flowbite (Inc. 2023) was chosen as it could be used as almost plug-and-play.

To ensure appropriate responsiveness of Lin $\TeX$, it was necessary to be able to introduce updates in real time. For this, HTMX (Software 2020) was utilised, as it is a tool that permits HTML tags other than the 'a' and 'form' tags to send HyperText Transfer Protocol (HTTP) requests to the server. For example, it is possible to use HTMX to create a HTTP request when a given key is pressed through its event management.

2.4 Guidelines for accessibility

The following key points have been used in the design of Lin $\TeX$ to ensure its accessibility, following WCAG 2.2 (W3C 2018) standards.

- The website should have all non-text content contain an alternative text (WCAG 2.2: Guideline 1.1).
- The website may be operable through the keyboard alone (WCAG 2.2: Guideline 2.1).
- LinT_EX should provide simple error messages.
- The website should allow for content to be distinguishable, including supporting resizing content and minimal colour usage (WCAG 2.2: Guideline 1.4).
- The website should be presentable in a simpler format if required (WCAG 2.2: Guideline 1.3).
- The website should include navigation features (WCAG 2.2: Guideline 2.4).
- The website should be predictable (WCAG 2.2: Guideline 3.2).
- The produced document by LinT_EX should contain appropriate tagging.
- The produced accessible document by LinT_EX should contain appropriate alternative text, actual text, or artifact.

2.5 Choosing a software development methodology

The available options under consideration for the project are the waterfall methodology, agile development methodology, test-driven development (TDD), and behaviour-driven development (BDD).

The waterfall methodology requires significant research early in the project to have success. However, as I lack experience in PEG script usage and many related topics for this project, it was likely that changes would be needed dynamically.

TDD was the next consideration, with a personal strong bias towards it due to its usefulness in being self-documenting and saving the hassle of writing the unit tests after creating the software, which can be a more draining process. However, as Pigeon, the PEG library chosen in choosing a PEG library under the Go programming language (section 2.2), generates Go files without a way to test individual components, it was not possible to create unit tests without it using complete L^AT_EX code for the feature that should be tested. Additionally,

creating unit tests can be time-consuming, which may be unsuitable for the short timeframe of the project.

BDD was removed from consideration for similar reasons as TDD, with the addition of it being more suitable for larger contexts such as games development, where it is a lot less practical to test every feature individually.

The final methodology, agile development methodology, allowed for code to be created quickly through its sprints. Additionally, as there was no prerequisite to create unit tests before creating the features, it saved a lot of time in development for the project. Consequently, agile development methodology was chosen for this project.

2.6 Screen reader choice

Through a small number of criteria, as showcased in Table 5, it was possible to reduce the choices for what screen reader to use in our context.

Table 5: Screen reader pricing status and OSes

Screen reader	Paid service	operating system support
JAWS	Yes	Microsoft Windows 10, 11, and server editions
Apple VoiceOver	No	Apple devices including iPhones, iPads, Macs, and AppleWatches
Windows Narrator	No	Windows 10 and 11
TalkBack	No	Android 6.0 Marshmallow onwards
ChromeVox	No	Primarily ChromeOS, with older versions supporting devices using Chrome browser
Supernova	Yes	Microsoft Windows 10, 11, server editions, and virtual environments
NVDA	No	Microsoft Windows 8.1 to 11

As a student, I do not have the leeway to purchase subscriptions for several screen readers, making those that require payments ruled out, even if it would just be in the form of a free trial. Additionally, there is an ethical dilemma in requiring payments for a necessary accessibility tool, which pushed me away from choosing them. Another criterion was that the Windows operating system (OS) should be supported, with emphasis placed on Windows 10 and Windows 11. These were chosen as they are the most common PC OS used, consequently targeting a larger base of users who may require screen readers.

Considering these criteria, NVDA and Microsoft Narrator were the final choices. When testing both on a placeholder document, emulating the experience of a screen reader user working with $\text{\LaTeX}$, I found that there was more difficulty with Windows Narrator in terms of having it read the correct extract on the document, often requiring using a mouse to get

to the correct location. Inversely, NVDA read the correct extract in full detail. Hence, by process of elimination to a degree, NVDA was the chosen screen reader for LinT_EX.

Adobe Acrobat Reader was used to work alongside NVDA to make the documents easier to navigate in more complex contexts, such as where the document may not be read as simply as top-to-bottom, left-to-right.

2.7 User study design choices

2.7.1 User study structure design

A combination of a think-aloud session and questionnaires was utilised in the creation of the user study.

A think-aloud session was performed where users think aloud while performing a given task, with the goal of identifying areas of improvement in the usability of the given context. This was utilised to acquire qualitative data regarding the usability of LinT_EX, with focus placed on most affected areas in Table 9.

It was deemed necessary to compare a L^AT_EX document before and after applying LinT_EX's changes to determine whether LinT_EX had a measurable impact on the screen reader accessibility of the documents. This comparison was created through a listening of the document's screen reader output at a given stage, and a questioning stage immediately after the relevant listening.

Additionally, it was possible that the participant's experience level in related topics would have an impact on their answers to questions or experience during the think-aloud session, consequently necessitating a small questionnaire to gauge experience levels.

2.7.2 User study question design

Factors that may influence a user's experience in the user study include their experience level in L^AT_EX, using screen readers, and listening to audio recordings of speaking. Likert scale data were used to gauge these experience levels in a numerical method, providing quantitative data.

Qualitative data were obtained from open-ended questions, with a clear focus placed on

each question, such that it would minimise confusion.

To gauge an understanding of how truly impactful an inaccessible document's impact is on a screen reader user's experience, participants will be questioned on what they could recall from the listening. However, as the participant will have worked with the document directly during the think-aloud session, it is redundant to include it in the final questioning.

It is necessary to understand what components of the relevant listening were found to be simple or difficult. However, these should be asked separately to ensure no confusion about what is being asked. The participant will be asked to provide their opinion on how well the screen reader audio output could be understood in terms of its contents, with a likert scale datum to provide quantitative data that can be analysed.

Participants were asked if they were likely to accommodate screen reader users in the future, to gain an understanding of whether LinTeX had made doing so appear to be an easier process.

2.8 GUI design considerations

LinTeX took strong inspiration from integrated development environments (IDEs) with regards to its user interface and user experience (UI/UX) design. This is because $\text{\LaTeX}$ in structure is similar to a programming language, even being Turing complete (Murrish 2012). Hence, an environment suitable for programming is suitable program $\text{\LaTeX}$ in, with some special considerations put aside for its quirks. For example, depending on the user's screen size and preferences, they may wish to have the output of the $\text{\LaTeX}$ documents taking up half of the screen, a separate browser window or monitor, or out of the way until compilation.

Additionally, online code editors such as LeetCode (Tang 2015), HackerRank (Ravisankar **and** Karunanidhi 2011), and CodeWars (Doctor **and** Hoffner 2012) were similar in nature to what LinTeX would like to achieve regarding its UI/UX, and were consequently used as inspiration.

3 Specification & Design

Describe all details of the design and procedures used to achieve the project objectives. Do this chronologically.

It should be detailed enough to allow for an assessment of the rigour of your process, and, in the case of research projects, in terms of how well grounded your research is in the research literature. In these cases, refer back to relevant sections in the previous chapter.

Say which software lifecycle approach you used e.g., Waterfall, Spiral, Agile.

How did you gather user requirements?

3.1 Methodology

Which methodology did you choose?

3.2 Analysis

How did you decide on the particular software artifact you decided to develop?

3.3 Requirements

Here you explain what the functional and non-functional requirements are. Explain how you prioritised them. See <https://www.nuclino.com/articles/functional-requirements> for more information.

Functional requirement: "The system must **do** [requirement]."

Non-functional requirement: "The system shall **be** [requirement]."

Well-written functional requirements typically have the following characteristics:

Necessary: Although functional requirements may have different priority, every one of them needs to relate to a particular business goal or user requirement.

Concise: Use simple and easy-to-understand language without any unnecessary jargon to prevent confusion or misinterpretations.

Attainable: All requirements you include need to be realistic within the time and budget constraints set in the business requirements document.

Granular: Do not try to combine many requirements within one. The more precise and granular your requirements are, the easier it is to manage them.

Consistent: Make sure the requirements do not contradict each other and use consistent terminology.

Verifiable: It should be possible to determine whether the requirement has been met at the end of the project.

3.3.1 Functional Requirements

This is the **WHAT** of your artifact.

Functional requirements are product features that developers must implement to enable the users to achieve their goals. They define the basic system behavior under specific conditions.

Functional requirements need to be clear, simple, and unambiguous. Examples:

- The system must send a confirmation email whenever an order is placed.
- The system must allow blog visitors to sign up for the newsletter by leaving their email.
- The system must allow users to verify their accounts using their phone number.

3.3.2 Non-Functional Requirements

This is the **HOW** of your artifact. Example non-functional requirement: “When the submit button is pressed, the confirmation screen must load within 2 seconds.”

3.4 Design

3.4.1 Interface Design

Explain how you used wireframes, and how you tested these to design the user interface.

3.4.2 System Design

Show how you designed your database (if appropriate) and how you designed your system architecture, and the individual parts. Use UML and an Entity Relationship diagram

4 Implementation

4.1 general overview of technologies used

4.2 website with server-client relationship

4.3 possibly include things about peg

4.4 verification & validation

- (manual testing and error messages for server-side errors that should not occur in most cases)

5 User study

A user study is an approach taken to understand the requirements of users in relation to your application (Center 2024). It can be used to better understand user requirements for those who will be using the system and consequently improve design decisions.

5.1 Study design

The user study was designed with the goal of evaluating the effectiveness of the tool, LinTeX, and identifying areas of improvement for the tool. To meet these goals, the user study was split into six key stages:

1. An introductory stage where the goal is to identify where the participant's knowledge lies in related topics.
2. An initial listening of the inaccessible document, which was produced for this user study.
3. A questioning stage based on the initial listening of the inaccessible document.
4. A think-aloud session for understanding the accessibility of LinTeX.
5. A final listening of the newly created accessible document by using LinTeX.
6. A questioning stage based on the final listening of the accessible document.

These stages allowed for a comparison of the before and after of using LinTeX, through stages 2 and 5, as well as associated questioning stages 3 and 6, allowing both qualitative and quantitative data to be obtained. Additionally, the think-aloud session provides an opportunity to judge the usability of LinTeX.

Finer details for the user study can be found in the user study script (section A.4), with the reasoning behind their choices located in the user study design choices (section 2.7).

5.2 Hypothesis

There were several hypotheses that this user study aimed to prove or disprove:

- H1** The accessible document created by LinTeX showed an improvement from the

inaccessible document initially used in terms of the document's accessibility for screen reader users.

H2 LinT_EX is reasonably accessible.

H3 LinT_EX provided useful error messages.

H4 The inaccessible document used in the initial listening is not memorable.

5.3 Results

The sample size for this user study was small, with 5 participants being recruited for it.

5.3.1 Memorable document extracts

The following table, namely Table 6, showcases the results from the question in the initial questioning (stage 3), which requests participants to summarise the document.

Table 6: Memorable aspects of document extracts

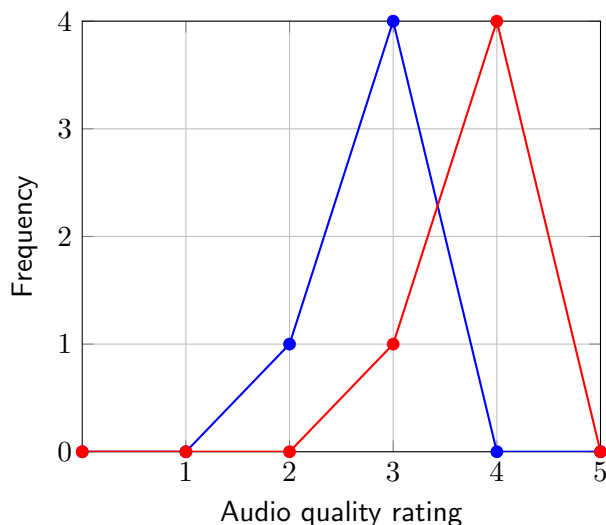
Document extract	Comment
Improving spoken skills	The majority of participants recalled spoken skills being improved through tongue twisters, yet there was minimal mentions of other techniques referenced in the inaccessible document.
House sale	A little over half of the participants had recalled a house being mentioned, with the currency and specific pricing often incorrect.
Favourite pet	All users recalled the extract about the cat being the favourite pet.
Muhammad and Noah name comparisons in Scotland	All users recalled this extract, with finer details such as: whose name was more popular, the positions of the names, and what names were in the context were often incorrect.
Grade comparisons	All users recalled this extract, with finer details such as: the friends' names, John's name, the class codes, whether the extract referenced John's friends or general students, who was assigned what grades, and the grades themselves, were often incorrect.

It should be noted that participants who had requested to pause or repeat a segment of the initial listening showcased a better recollection of the document extracts, with the average participant able to recall 4 extracts on average.

5.3.2 Accessible document and inaccessible document comparisons

The following figure, namely Figure 1, showcases the comparison of Likert scale data obtained from the user study that queries the participants about the quality of the screen reader's audio output regarding its contents. This data was used to create a frequency polygon graph, for which the blue line showcases the data from the initial questioning (stage 3) and the red line showcases the data from the final questioning (stage 6).

Figure 1: Comparing screen reader output quality ratings before and after applying LinTeX



The key difficult components of both the initial listening (stage 2) and final listening (stage 5) can be found in Table 7. It should be noted that this is not an exhaustive list of all difficult components from both stages, but rather the key points that a majority of participants in the user study emphasised. For example, a few participants noted that the scale of numbers included in a table had an impact on its legibility.

Table 7: Difficult aspects from the initial and final listenings

Initial document	Final document
Repetition of 'percent' in the second table was found to be distracting.	Repetition of 'percent' in the second table was found to be distracting.
Images were difficult to keep track of when and where they had occurred.	Tables contained too much metadata, with focus placed on 'column N' being repeated for each table cell.
Images being plain file names made it difficult to ascertain their purpose.	Document metadata could be confused with the contents of the document, with emphasis placed on column numbers being mixed with name positions in the first table.
Headings and pages did not have sufficiently long pauses before beginning.	
Tables were read too fast with no pauses between table cells.	
The table containing names is viewed from top to bottom, whereas the screen reader reads from left to right.	

The key simple components of both the initial listening (stage 2) and final listening (stage 5) can be found in Table 8. It should be noted that this is not an exhaustive list of all simple components from both stages, but rather the key points that a majority of participants in the user study emphasised. For example, many participants enjoyed the inclusion of an

example tongue twister in place of the relevant image.

Table 8: Simple aspects from the initial and final listenings

Initial document	Final document
Standalone sentences were read slower and with pauses.	Standalone sentences were read slower and with pauses.
Sentences separated by images or tables were easier to distinguish.	Sentences separated by images or tables were easier to distinguish.
Lists provided in bullet point form were simple to understand due to a clear starting point.	Lists provided in bullet point form were simple to understand due to a clear starting point. Each point in the list started with 'bullet' instead of 'bullet point' which was the case previously, consequently speeding up the screen reader's output.
The table discussing grades is read left to right, whereas the table discussing names is read top to bottom. When the table can be viewed left to right, it followed the same order the screen reader used when creating the audio output.	The table discussing grades is read left to right, whereas the table discussing names is read top to bottom. When the table can be viewed left to right, it followed the same order the screen reader used when creating the audio output.
Supplementary sentences included before or after images and tables improved the overall clarity of the related extracts.	Supplementary sentences included before or after images and tables improved the overall clarity of the related extracts.
	Images were easier to understand due to their included descriptors.
	The screen reader included more pauses and spoke slower, with emphasis placed on tables.
	Structural indicators such as 'out of list', 'out of table', and 'table with M columns and N rows' improved tracking of where the screen reader was on the document at that moment.
	Reading the table heading before each table data cell improved tracking of what data correlated to what name in the first table.

It should be noted that all participants required less pausing or repetition of an extract in the final listening (stage 5) compared to the initial listening (stage 2). Additionally, another point of note is that tables were a frequent aspect that required pausing or repetition.

5.3.3 Applying student's paired t-test

The student's paired t-test was developed by William Sealy Gosset in 1908 (Wadhwa and Marappa-Ganeshan 2023). Its purpose, in our context, is to determine if the difference between our audio output quality Likert scale data in Figure 1 is significant.

Figure 2: Paired t-test formula

$$t = \frac{\bar{d} - \mu_0}{\frac{s_d}{\sqrt{n}}} \quad (1)$$

Figure 2 has a standard deviation (s_d) of 0, as the difference between all results was 1. As

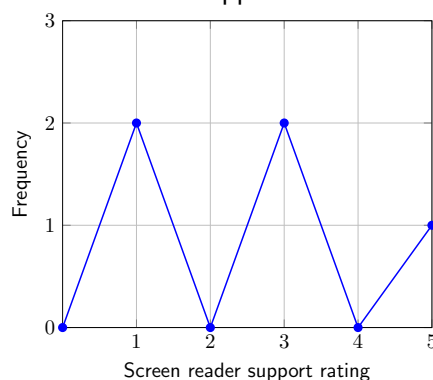
a result of this, it leads to a division by zero. Depending on interpretation, this could be seen as t approaching infinity (∞) or simply an undefined result.

Under the basis of t approaching infinity, we can consequently view it as a large t value, which suggests there is statistical significance in the results.

5.3.4 Likelihood to support screen reader users

The following figure, namely Figure 3, showcases the results from the Likert scale data obtained in the final questioning (stage 6), which queries participants on their likelihood to support screen reader users in the future.

Figure 3: Participants likelihood to support screen reader users in the future



5.3.5 LinTeX usability focuses

The following table, namely Table 9, highlights areas where usability could be improved or was found largely useful during the think-aloud session (stage 4). It should be noted that some results that stemmed from a lack of LaTeX knowledge were not included, as Table 9 aimed to identify the key components of usability.

5.4 Analysis

5.4.1 Reasoning behind document extracts being memorable

This section uses data from Table 6 to make an analysis of the reasoning behind why specific sections were memorable, and others were not.

While the tongue twister was memorable in the initial listening (stage 2), likely due to it

Table 9: LinTeX usability

Affected area	Comment
Uploading a file	Lacked responsiveness regarding actions performed when the upload button was clicked without a selected file, and informing the participant that a file has been uploaded.
Button phrasing	Phrasing of 'generate output' button caused participants to assume it could only be clicked after all errors were fixed.
Scrolling options	Scrolling options for the in-built code editor, errors, and the website as a whole, made navigation more difficult.
Submitting a document	A keybind to generate the document's PDF output was requested.
Error extracts	Most error extracts, the code taken from where the error had occurred, were helpful. However, a few syntax errors, such as a missing closing curly brace only provided the opening curly brace within its extract. It should be noted that many users utilised the browser's search features to locate where the error had occurred.
Image error messages	No difficulties found.
Table error messages	Attempts were made to place the 'tabular' group within the required argument of '\tagpdfsetup'.
Tagging error messages	Uncertainty existed with what should be provided to the required argument of '\DocumentMetadata' and how to handle several key-value pairs being provided to it, with some attempts to add an extra required argument.
Error messages	Examples helped to achieve the correct output, with long error messages found from requesting more details alleviating many prior concerns.
Text size	Text size was small.
Colour scheme	The colour scheme of the website may be inaccessible for users with dyslexia.

having a relatively large extract focused on it, participants had noted that providing the actual tongue twister made for a more enjoyable and consequently memorable experience in the final listening (stage 5).

Participants frequently misremembered finer details in smaller extracts relating to images, with only the extract on the cat being the favourite pet being the most memorable, likely due to its simplicity. This could be biased by the participants not being informed what specific questions would be asked, or the question being asked after the full document has been read, consequently requiring the participants to keep the full document in mind by that point. The latter point highlights a struggle a screen reader user may face.

Regarding name positioning and grade comparisons, a reasonable assumption as to why the overall topics were memorable despite finer details being frequently incorrect is that they required a greater degree of focus due to having more data in the tables. However, due to the magnitude of the data, and the speed and lack of pauses in their listening, as referenced in Table 7, the particulars of the data were often mistaken. Additionally, supplementary sentences surrounding these tables often provided a general overview of what the tables had contained, which may have influenced a participant's understanding.

5.4.2 LinTeX's impact on document screen reader accessibility

Figure 1 showcases that the accessible document's audio output rating had a higher peak compared to the inaccessible document. Additionally, we can view that the accessible document is shifted one data point higher on the x-axis at all points, with the same overall shape maintained as the inaccessible document. We can consequently conclude that, based on this quantitative data, LinTeX had provided an improvement to the document's accessibility, with emphasis placed on screen readers.

Table 7 highlights the components of the initial listening (stage 2) and final listening (stage 5) that were difficult to understand. This permits a comparison between the two columns, for which we can determine that the majority of difficult aspects had been improved, as they were no longer in the final listening (stage 5)'s column. However, there remained two components that provided some confusion in the final listening (stage 5):

- The document's contents added unnecessary confusion at times, where it could be confused for metadata that the screen reader was reading out.
- The tables within the document contained too much metadata. The phrasing 'less is more' could be applied here, where too much information obstructed a participant's ability to understand its contents.

Table 8 showcases the fact that there were many more simple to understand components in the accessible document compared to the inaccessible document, with many of the previously helpful components remaining the same or having mild improvements. Consequently, we can deduce that simple components were retained with the same, or arguably better, quality.

5.4.3 Areas of improvement for LinTeX usability

Table 9 highlights many areas where LinTeX suffers in its usability. Hence, some proposed solutions for these concerns have been indicated below:

- The file upload process could be simplified by combining both the file selector and the upload file button. This would reduce the number of steps required on the user's part to upload the file and begin using LinTeX.

- The phrasing of specific website components could be changed. For example, ‘generate output’ could be swapped out for ‘check document’ which has different connotations of applying some form of validation to the document. Additionally, a keybind to shortcut clicking this button has been indicated to be a desired feature, due to the reliance on frequently clicking it.
- Scrolling within the webpage could be alleviated by removing scrolling of the full webpage (for the tool webpage). That would ensure there is always a ‘check document’ button at the bottom, while still allowing the flexibility of scrolling error messages or the code editor.
- More information should be provided by some error messages, with emphasis on those related to tables or tagging. By providing more information upfront, there will be less confusion about where to progress from a specific point.
- The error extracts being less helpful in the context of syntax errors should be alleviated by providing the line and character position information. In the future, if there is enough time, an automated fix feature could be implemented.
- Text sizes being too small can be solved by simply increasing their size. There is also potential for choosing different font families that fit the context better.
- The colour scheme was indicated to have not been distinct enough for dyslexia users, which can be alleviated by choosing and applying a more accessible colour scheme for this user group.

5.5 Hypothesis testing and discussion

It can be claimed that, as there is statistical significance in applying student’s paired t-test (section 5.3.3), a right-tailed test can be used to ascertain that there is a definitive improvement in the screen reader accessibility of the documents after applying LinTeX. Additionally, based on section 5.3.2, it can be concluded that there was a significant improvement in the document’s screen reader accessibility after applying LinTeX, as it showed previously difficult aspects becoming simpler to understand, with minimal difficult aspects in the accessible document. Consequently, we choose not to reject the

hypothesis **H1**.

The hypothesis **H2** has been rejected in our case, due to the magnitude of accessibility-related difficulties found in the Table 9. However, proposed solutions have been provided in areas of improvement for LinT_EX usability (section 5.4.3), which indicate processes that may be undertaken to solve many of the identified concerns.

It can be reasonably chosen to not reject the hypothesis **H3** as the majority of error messages were greatly useful based on Table 5.4.3, with only a few points made to provide more information upfront to avoid the user being at a standstill for what action to perform at a given point and providing more information regarding the location the error had occurred.

The hypothesis **H4** is arguable, as the main key points were often recalled by the participants. However, as many finer details were significantly wrong, the overall memorability of the inaccessible document's content could be considered low. Consequently, this would permit not rejecting the hypothesis **H4**.

6 Discussion and reflection

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS.

Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

6.1 something

should've put more emphasis on researching frontend

7 Conclusion

7.1 Project summary

7.2 Project limitations

7.3 Achievements – potentially optional/swap with final thoughts or another area?

7.4 Future work

7.5 Reflection

7.6 Final thoughts

The conclusion is similar to when a plane lands. You don't rewrite the introduction. You say something like - I addressed the problem outlined in the introduction, and I built some software to do this. I tested the software like this ADD FEW WORDS.

Summarize main findings drawn from the project work. Mention the objectives or research questions. Do not repeat points raised in the discussion and reflection Section. If applicable, you can make recommendations. The conclusion should NOT contain any references.

References

- Authors, The go-latex (2020). *latex*. English. <https://pkg.go.dev/codeberg.org/go-latex/latex>. URL: <https://pkg.go.dev/codeberg.org/go-latex/latex>.
- Authors, The Bootstrap (2011). *Bootstrap*. English. <https://getbootstrap.com/>. URL: <https://getbootstrap.com/>.
- Authors, The Go (2009a). *Exec*. English. <https://pkg.go.dev/os/exec>. URL: <https://pkg.go.dev/os/exec>.
- (2009b). *html/template*. English. <https://pkg.go.dev/html/template>. URL: <https://pkg.go.dev/html/template>.
- (2009c). *net/http*. English. <https://pkg.go.dev/net/http>. URL: <https://pkg.go.dev/net/http>.
- Authors, The Gorilla (2023). *gorilla/mux*. English. <https://pkg.go.dev/github.com/gorilla/mux>. URL: <https://pkg.go.dev/github.com/gorilla/mux>.
- Center, Delaware Government Information (2024). *What is User Research and Why Should it be a Part of Your Process?* English. URL: <https://gic.delaware.gov/what-is-user-research-and-why-should-it-be-a-part-of-your-process/>.
- Doctor, Nathan **and** Jake Hoffner (2012). *CodeWars*. English. <https://www.codewars.com/>. URL: <https://www.codewars.com/>.
- Exeter, University of (2025). *Test Statistics*. English. URL: <https://libguides.exeter.ac.uk/c.php?g=729256&p=5313318>.
- Inc., Bergside (2023). *Flowbite*. English. <https://flowbite.com/>. URL: <https://flowbite.com/>.
- Kpym (2020). *latex-fast-compile*. English. <https://pkg.go.dev/github.com/kpym/latex-fast-compile>. URL: <https://pkg.go.dev/github.com/kpym/latex-fast-compile>.
- Martínez-Almeida, Manuel (2014). *Gin Web Framework*. English. <https://pkg.go.dev/github.com/gin-gonic/gin>. URL: <https://pkg.go.dev/github.com/gin-gonic/gin>.
- Murrish, Robert (2012). *?LaTeX is More Powerful than you Think - Computing the Fibonacci Numbers and Turing Completeness?* English. **in** URL: https://www.overleaf.com/learn/latex/Articles/LaTeX_is_More_Powerful_than_you_Think_-_Computing_the_Fibonacci_Numbers_and_Turing_Completeness.
- National Library of Medicine, Network of the (2026a). *Qualitative Data*. English. URL: <https://www.nlm.gov/resources/data-glossary/qualitative-data>.
- (2026b). *Quantitative Data*. English. URL: <https://www.nlm.gov/resources/data-glossary/quantitative-data>.
- Ravisankar, Vivek **and** Harishankaran Karunanidhi (2011). *HackerRank*. English. <https://www.hackerrank.com/>. URL: <https://www.hackerrank.com/>.
- Software, Big Sky (2020). *HTMX*. English. <https://htmx.org/>. URL: <https://htmx.org/>.
- Sullivan, Gail M. (2013). *?Analyzing and Interpreting Data From Likert-Type Scales?* English. **in** URL: <https://pmc.ncbi.nlm.nih.gov/articles/PMC3886444/>.
- Tailwind Labs, Inc. (2017). *Tailwind CSS*. English. <https://tailwindcss.com/>. URL: <https://tailwindcss.com/>.
- Tang, Winston (2015). *Leetcode*. English. <https://leetcode.com/>. URL: <https://leetcode.com/>.

- W3C (2018). *Web Content Accessibility Guidelines (WCAG) 2.2*. <https://www.w3.org/TR/WCAG22/>. A W3C Recommendation. URL: <https://www.w3.org/TR/WCAG22/>.
- Wadhwa, Raoul R. **and** Raghavendra Marappa-Ganeshan (2023). *?T Test?* English. **in** URL: <https://www.ncbi.nlm.nih.gov/books/NBK553048/>.

A Appendix

This is where you can include your documentation.

Remember that the marker is not required to read this, but might well check to ensure that you have included product documentation, and ethical approval, as required.

A.1 Ethical approval form

If your project required you to do any evaluation with humans, you **MUST** include this. It can be downloaded from the Ethics system.

<https://local.cis.strath.ac.uk/wp/extras/ethics/index.php>

A.2 Participant information sheet

If your project required you to do any evaluation with humans, you **MUST** include this

<https://www.strath.ac.uk/ethics/information-sheet-and-consent-form/>

A.3 Consent form

If your project required you to do any evaluation with humans, you **MUST** include this.

<https://www.strath.ac.uk/ethics/information-sheet-and-consent-form/>

A.4 User study script

something